

MusicGO

Guía de Defensa — Fase 2

Cátedra: Programación Orientada a Objetos · UCAB

Profesor: Marcel J. Castro G.

Equipo: Equipo MusicGO

Defensa grupal: 03-07-26 · **Defensa individual:** 10-07-26

1. Guion de la demostración (defensa grupal)

Sigue estos pasos en orden; cubren todos los requerimientos del enunciado.

0. Arranque: `./mvnw javafx:run` . Aparece el login.

1. Admin: entra como `admin / admin123` . Crea una canción nueva (marca clasificación *Explícito* en una para la demo de control parental), edita una y elimina otra. Resalta que todo se guarda solo (en JSON, en hilo de fondo).

2. Tiempo real: cierra sesión y entra como usuario `rocker99` . Muestra que la canción creada por el admin **ya aparece** en el catálogo.

3. Explorar: usa los filtros (Canciones / Podcasts / Productos) en la cuadrícula.

4. Reproducir: dale play a una canción; observa la barra de progreso y los metadatos.

5. Control parental: entra como `podcastlover` (lo tiene activo) e intenta reproducir contenido explícito → aparece la advertencia que bloquea la reproducción.

6. Playlists: crea una playlist, agrégle audios desde el catálogo, quita uno.

7. Compartir: comparte la playlist con otro alias; inicia sesión como ese usuario y muéstrala en "Compartidas conmigo".

8. Pagos: compra un producto probando los **tres medios**: tarjeta (valida formato), transferencia (valida cuenta) y saldo virtual. Provoca un error (tarjeta inválida o fondos insuficientes) para mostrar la alerta controlada.

9. Dashboard: abre Estadísticas; muestra el Top de más reproducidos y recarga saldo.

2. Mapa de la arquitectura (qué archivo hace qué)

Requerimiento	Archivos clave
Entrada / navegación	MusicGoApp , app/AppContext , app/Router
Login y perfiles	vista/LoginVista , controlador/LoginControlador , servicios/GestorAutenticacion
Back-office (CRUD)	vista/admin/* , controlador/AdminControlador , servicios/GestorCatalogo
Pagos (ISP/Strategy)	interfaces/pago/* , modelo/pago/* , servicios/pago/ProcesadorPago
Control parental	modelo/Clasificacion , servicios/GestorReproduccion , excepciones/ContenidoRestringidoException
Compartir playlist	servicios/GestorPlaylists , controlador/PlaylistControlador
Dashboard / Top	servicios/GestorEstadisticas , vista/usuario/DashboardVista
Persistencia en hilos	persistencia/ServicioPersistencia , persistencia/Mapeador*

3. SOLID en el código (defensa individual)

Si el profesor pide señalar cada principio, usa estas respuestas y abre el archivo.

SRP — ¿una clase, una responsabilidad?

Cada gestor hace una sola tarea. Ej.: `GestorAutenticacion` solo valida credenciales; el mapeo JSON vive aparte en los `Mapeador*`, no en el gestor.

OCP — ¿abierto a extensión, cerrado a modificación?

Para añadir un medio de pago creo una clase que implemente `MedioPago`; `ProcesadorPago` no se toca. Igual con un tipo de audio nuevo: extendiendo `Audio` y el reproductor sigue igual (polimorfismo).

LSP — ¿los subtipos sustituyen al base?

`FondosInsuficientesException` extiende `PagoRechazadoException`: el controlador puede capturar la base y el flujo funciona con cualquier subtipo.

ISP — ¿interfaces pequeñas?

La pasarela usa `RequiereTarjeta`, `RequiereCuentaBancaria` y `UsaSaldoVirtual`. `SaldoVirtual` no arrastra métodos de tarjeta; nadie implementa métodos vacíos.

DIP — ¿se depende de abstracciones?

Los controladores reciben los servicios desde `AppContext` (composition root). Dependen de las clases de servicio/abstracciones, no construyen sus propias dependencias.

4. Pilares de la POO

Pilar	Ejemplo en MusicGO
Abstracción	Clases abstractas <code>Audio</code> y <code>Producto</code> ; interfaces <code>Reproducible</code> , <code>Comprable</code> , <code>MedioPago</code> .
Encapsulamiento	Atributos privados con getters/setters validados; <code>Administrador</code> oculta la clave y solo expone <code>credencialesValidas()</code> .
Herencia	<code>Cancion</code> y <code>EpisodioPodcast</code> heredan de <code>Audio</code> ; <code>ArteVisualAlbum</code> y <code>PaqueteTopTen</code> de <code>Producto</code> .
Polimorfismo	<code>creditos()</code> y <code>detalle()</code> se resuelven según el tipo concreto; cada <code>MedioPago</code> ejecuta su propio <code>procesar()</code> .

5. Prueba de modificación "en caliente"

El profesor puede pedir un cambio modular en vivo. Aquí está dónde tocar:

Si te piden...	Haces...
"Agrega un medio de pago (ej. PayPal)"	Nueva clase en <code>modelo/pago/</code> que implemente <code>MedioPago</code> (y su interfaz específica si aplica) + una opción en <code>DialogoPago</code> . Nada más.
"Agrega un atributo a Canción"	Campo + getter/setter en <code>Cancion</code> , leer/escribir en <code>MapeadorCatalogo</code> , y el campo en <code>DialogosCatalogo</code> .
"Agrega una pantalla nueva"	Nueva <code>*Vista</code> (solo UI) + nuevo <code>*Controlador</code> ; enlázala en el coordinador y el <code>Router</code> .
"Cambia la estética"	Solo <code>resources/.../estilos.css</code> .
"Agrega un nivel de clasificación"	Una constante en el enum <code>Clasificacion</code> .

6. Preguntas frecuentes

¿Por qué JavaFX en código y no FXML?

Para tener control total y mantener el MVC con una fábrica de componentes reutilizable; las vistas siguen sin lógica de negocio.

¿Cómo se actualiza la app del usuario cuando el admin cambia el catálogo?

Con un patrón observador en `GestorCatalogo` basado en `Runnable` (sin acoplar el modelo a JavaFX); el controlador del usuario se suscribe y refresca con `Platform.runLater`.

¿Dónde están los hilos de persistencia?

En `ServicioPersistencia`: un `ExecutorService` de un hilo daemon serializa las escrituras y cierra limpio al salir.

¿Por qué no se rompe la app cuando algo falla?

Las reglas violadas lanzan excepciones personalizadas que el controlador captura con `try-catch` y muestra como `Alert`, sin tumbar el hilo de JavaFX.

¿Cómo garantizan la modularidad?

72 archivos, ninguno supera 300 líneas; una responsabilidad por clase.

7. Checklist de requerimientos cumplidos

- ✓ GUI JavaFX con login y dos interfaces independientes (admin / usuario).
- ✓ Back-office con CRUD de catálogo y actualización en tiempo real.
- ✓ Registro/login de usuario con validación de unicidad en vivo.
- ✓ Exploración visual en cuadrícula que diferencia audios de productos.
- ✓ Gestión avanzada de playlists con controles interactivos.
- ✓ Reproductor gráfico con barra de progreso y metadatos.
- ✓ Compartir Playlist por alias.
- ✓ Filtro y restricción de contenido (control parental + advertencia).
- ✓ Dashboard dinámico con Top de más reproducidos.
- ✓ Pasarela de pagos multimodal con interfaces segregadas (ISP).
- ✓ Excepciones personalizadas capturadas en el controlador.
- ✓ Persistencia JSON inmediata con hilos.
- ✓ MVC estricto, SOLID, KISS y modularidad (≤ 300 líneas/archivo).